Intraday Risk Metric Recalculation

Reusable analytical risk kernels with frozen covariance



Executed research outcome. Fifty distinct risk expressions plus one immutable revision, 20,000 completed synthetic market updates, 24,167 scalar paceval evaluations, 1,600 packed batch checks and 3,721 numerical surface points.

Scope. A deliberately bounded, frozen-covariance analytical research workflow. Not a bank-approved market-risk system, a measured real-time engine or regulatory expected shortfall.

Cover is a conceptual image, not a numerical research result.

1. Industrial context and current practice

Banks and investment firms monitor portfolio factor exposures and risk limits as positions and market prices change during a trading day. A typical production chain combines position capture, market data controls, valuation, approved sensitivity models, risk aggregation and exception processing. The mathematical kernel here is only the risk aggregation calculation.

Example operational workflow

A risk desk receives approved, dimensioned position sensitivities, obtains a previously calibrated covariance matrix, evaluates a risk metric, checks an internal research limit, and records changes across market updates. This demonstrator executes that repeatable mathematical subfunction with paceval while all approvals and risk-limit routing remain in the C++ host.

What current practice does, and does not, imply

Classical RiskMetrics-style delta-normal measurement uses the quadratic portfolio variance form. Regulatory market-risk internal-model approaches under the Basel Framework may instead require expected shortfall, backtesting, model approval, and detailed risk-factor treatment. The Basel Core Principles emphasize market-risk limits, independent controls, and reliable valuation data. Therefore the present metric must not be labelled regulatory expected shortfall or an approved capital number.

Research question

Can fifty controlled mathematical risk expressions, using a frozen covariance input and synthetic intraday position updates, be registered as persistent paceval computation objects and evaluated repeatedly while retaining transparent independent numeric checks, scalar and batch execution, and explicit formula versioning?

Sources: RiskMetrics Technical Document (1996); BIS MAR30 (updated July 2024); BIS Basel Core Principles (April 2024); paceval public C/C++ API. Full links appear on page 10.

2. Defined mathematical model

The signed position exposures w1 and w2 are in EUR millions. Covariance entries describe dimensionless daily factor returns; thus the risk measure is in EUR millions. The synthetic shock s is dimensionless. The covariance matrix is constant for every experiment, and it is positive definite:
det(Sigma)=0.0002*0.00045-0.00012^2=7.56e-8 > 0.

Frozen covariance and the baseline

```
Sigma = [[0.00020, 0.00012], [0.00012, 0.00045]]
```

Fixed, internally consistent 2 x 2 factor-return covariance matrix.

```
q = 0.00020*w1^2 + 0.00024*w1*w2 + 0.00045*w2^2
```

Two-factor quadratic portfolio variance.

```
R0 = sqrt(q)
```

Delta-normal-style portfolio standard-deviation metric; not Value at Risk.

```
R1 = sqrt(q)*(1 + a*s)
```

Multiplicative synthetic shock to the baseline risk metric.

```
R2 = sqrt(q + 0.00002*a*(w1^2+w2^2)*s^2)
```

Illustrative shock-dependent variance augmentation.

```
R3 = sqrt(0.0002*w1^2 + 0.00045*w2^2 + 2*(0.00012+0.00002*a*s)*w1*w2)
```

Bounded synthetic covariance sensitivity.

```
R4 = sqrt(q) + 0.004*a*sqrt(w1^2+w2^2)*sqrt(1+s^2)
```

Illustrative concentration-type, non-negative risk add-on.

```
R5 = sqrt(q) + 0.00015*a*abs(w1*w2)*s^2 + 0.002*b*sqrt(1+s^2)
```

Nonlinear two-factor interaction add-on.

Fifty formulas vary a and b across five families. Bounds on signed positions, shock size and coefficients ensure the expressions are finite and their square-root radicands nonnegative for the defined workload.

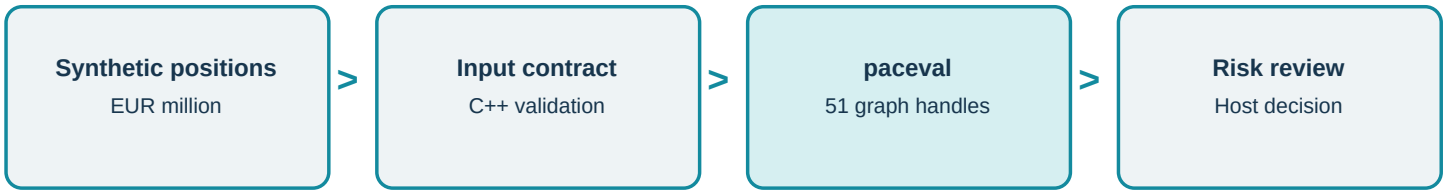
3. paceval architecture and responsibilities

What the paceval runtime calculates

The original SDK parses each formula into a computation object using paceval_CreateComputation. Repeated scalar evaluation uses paceval_dGetComputationResult. Separate packed-batch validation uses paceval_dGetComputationResultExt. The mathematical handles are created once and deleted on application shutdown.

Which operations remain outside paceval

The C++ host controls deterministic pseudorandom inputs, bounded-value checks, simulation ordering, formula-selection policy, synthetic limits, CSV export and independent reference arithmetic. It does not estimate a covariance matrix, fit a distribution, perform VaR backtesting, price options or connect to a live exchange.



Immutable mathematical revision

The fifty base formulas remain registered throughout the experiment. A different coefficient creates a 51st calculation object. The old and revised expressions are evaluated side by side, showing that a controlled formula update does not rewrite earlier computations.

Analytical model interpretation

Risk figures are simulated pointwise outputs of specified formulas. In real trading-book risk management, valuation controls, data lineage, governance, market calibration, sensitivity generation, approved limits and independent validation are additional essential tasks.

4. Executed workload and numerical assurance

The complete deterministic experiment uses a fixed random seed (20261112), finite synthetic position inputs, fifty formula variants and an additional immutable revision. Scalar checks compare selected calls with a separately implemented C++ reference; the released sampled data undergo an independent 80-digit mpmath audit.

Reproducible evidence from the paceval SDK

Original mathematical expression variants	**50**
Persistent graphs including controlled revision	**51**
Completed synthetic intraday updates	**20,000**
Scalar SDK evaluations	**24,167**
Packed batch checks	**1,600**
3D surface sample points	**3,721**
Independent 80-digit numerical sample checks	**500 / 500 passed**
Invalid input contract checks	**3 / 3 passed**
CMake / CTest integration	**1 / 1 passed**

The experiment tests graph reuse and the mathematical workflow, not processing-speed superiority or p99.9 latency. The recorded count of formula evaluations includes the replay, numerical surface, stress curves and formula-revision experiments. Released audit files contain detailed numerical checks; this public research report does not compare C++ and paceval numerical errors.

The resulting risk levels are neither estimates of future portfolio losses nor calibrated regulatory capital requirements. Intraday risk limits and numerical thresholds in this experiment are explicitly synthetic.

5. Intraday replay and rule distributions

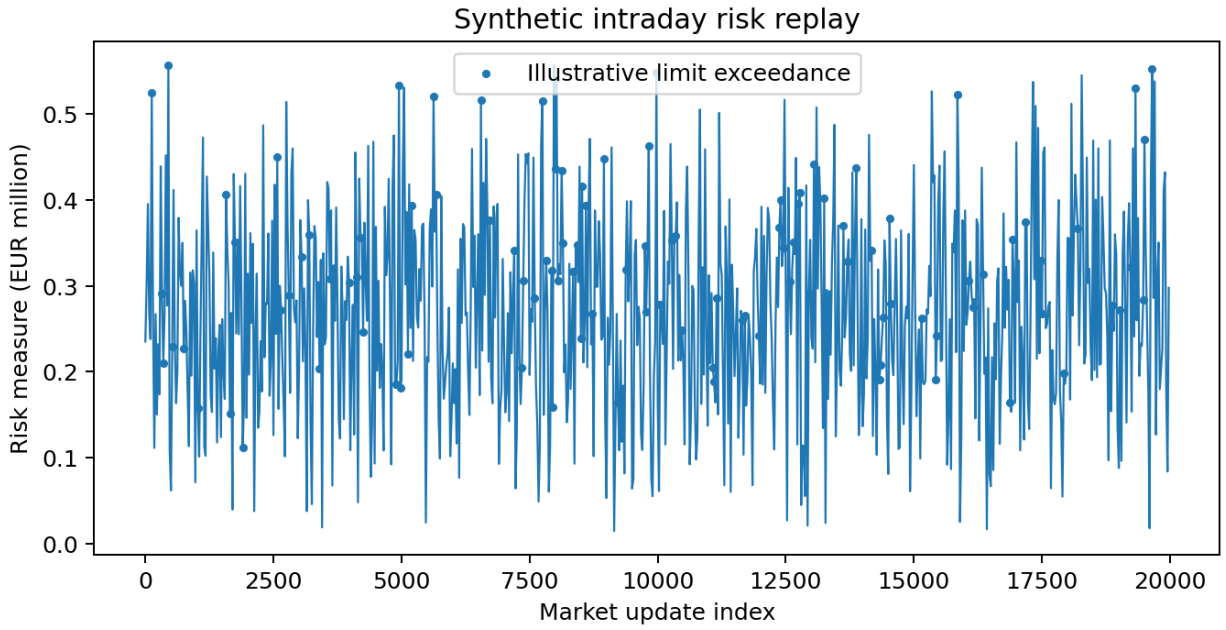


Figure. Synthetic replay sampled every 25th update; colored points show illustrative host alerts.

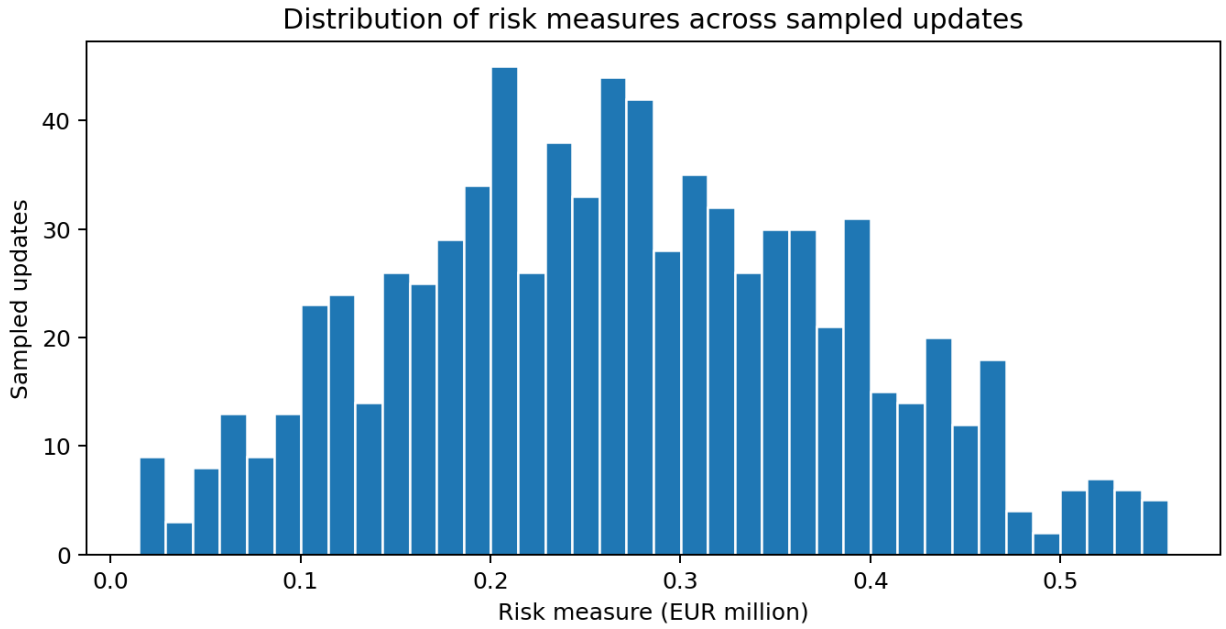


Figure. Distribution of sampled portfolio risk measure values.

6. Risk rules and portfolio factor sensitivity

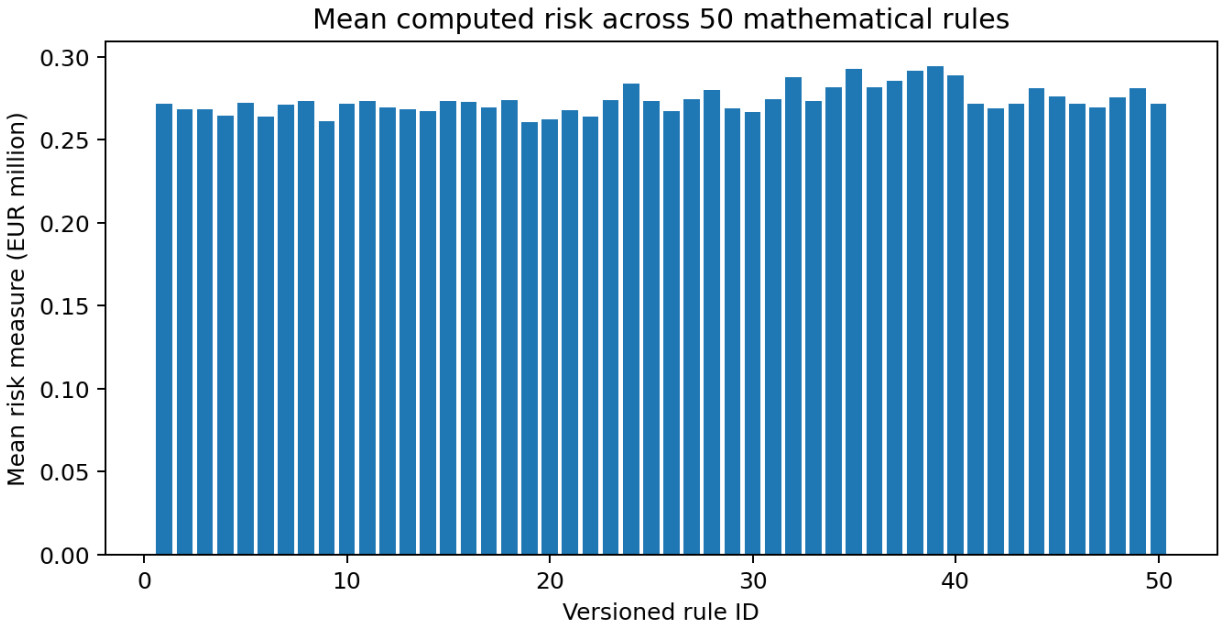


Figure. Different analytical rule variants yield differing research risk levels.

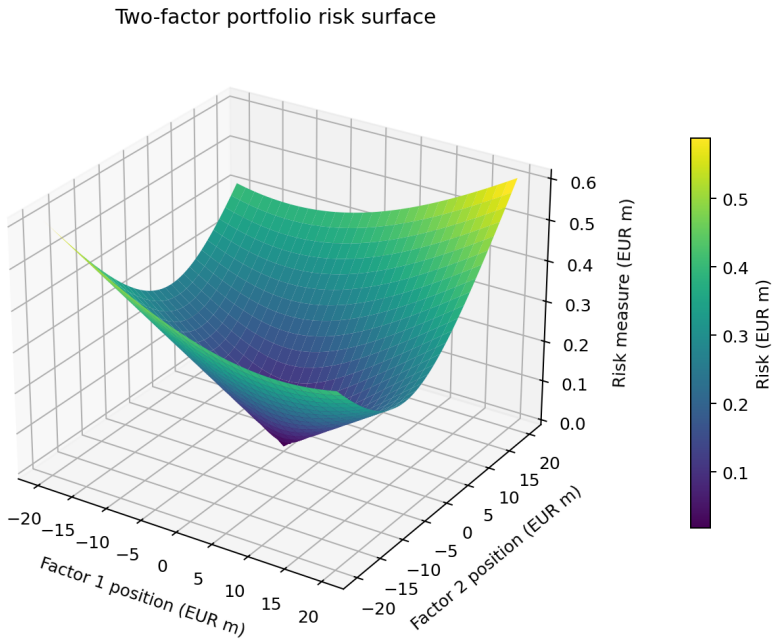


Figure. 3D risk surface from 3,721 actual paceval values with frozen covariance.

7. Risk contours and stress scenarios

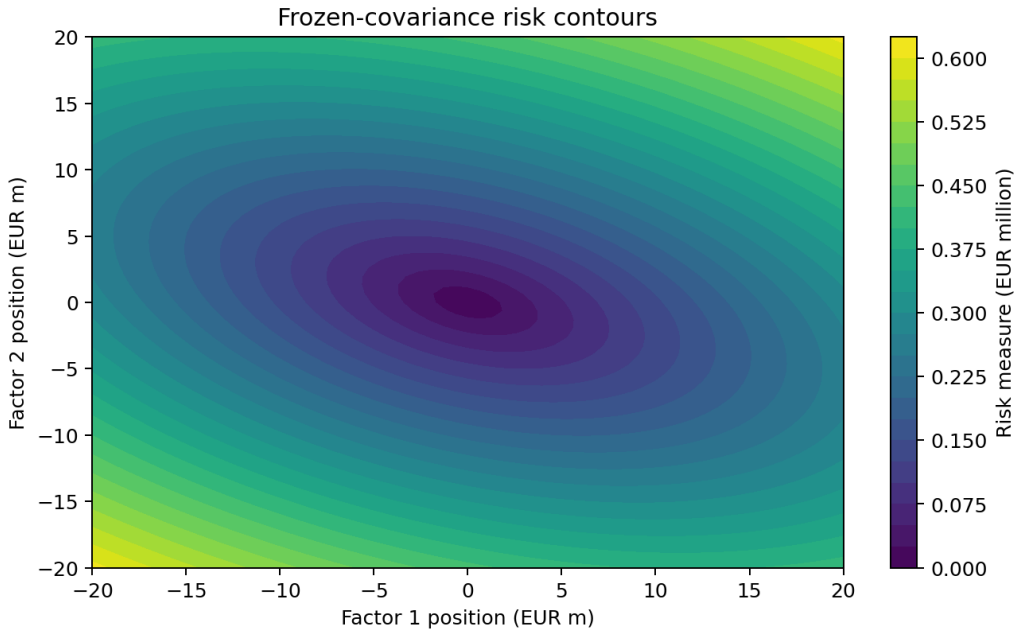


Figure. Two-factor risk contours use the same computed 3D grid.

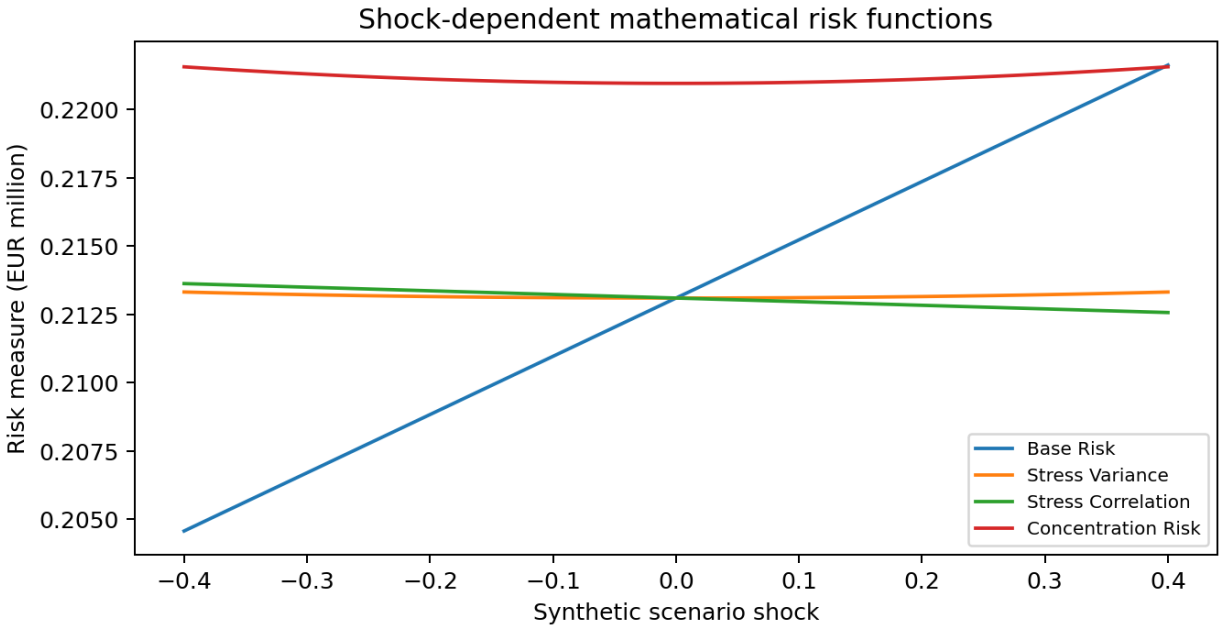


Figure. Four model families evaluated over a bounded synthetic shock range.

8. Immutable revisions and risk-limit examples

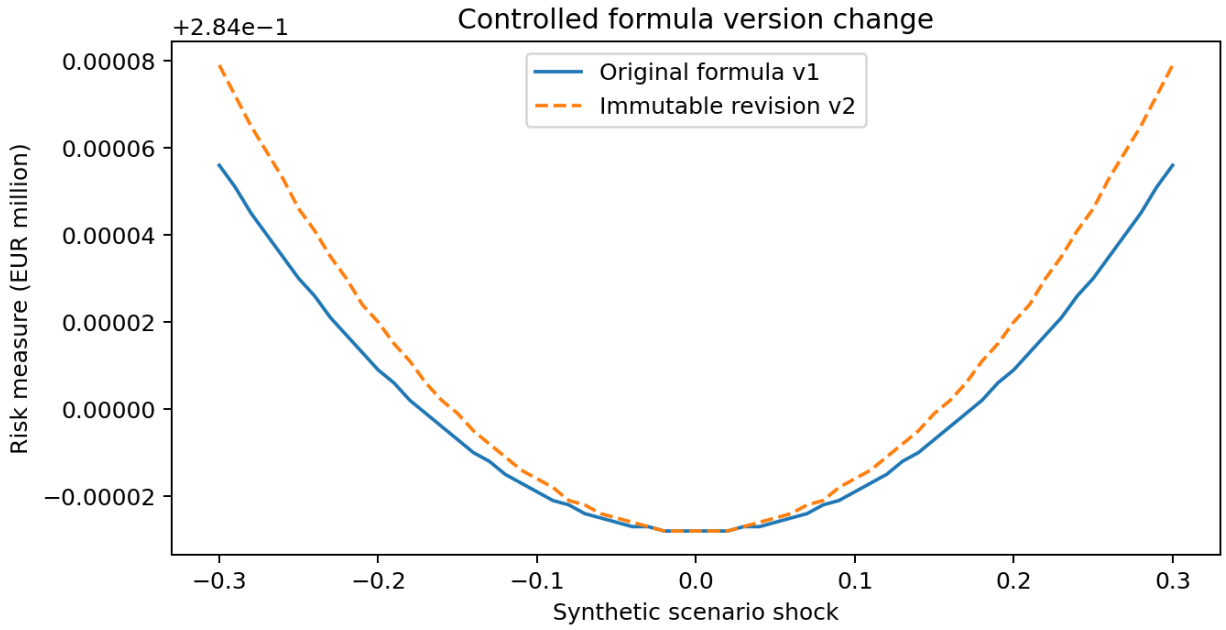


Figure. The old and revised formulas remain individually callable.

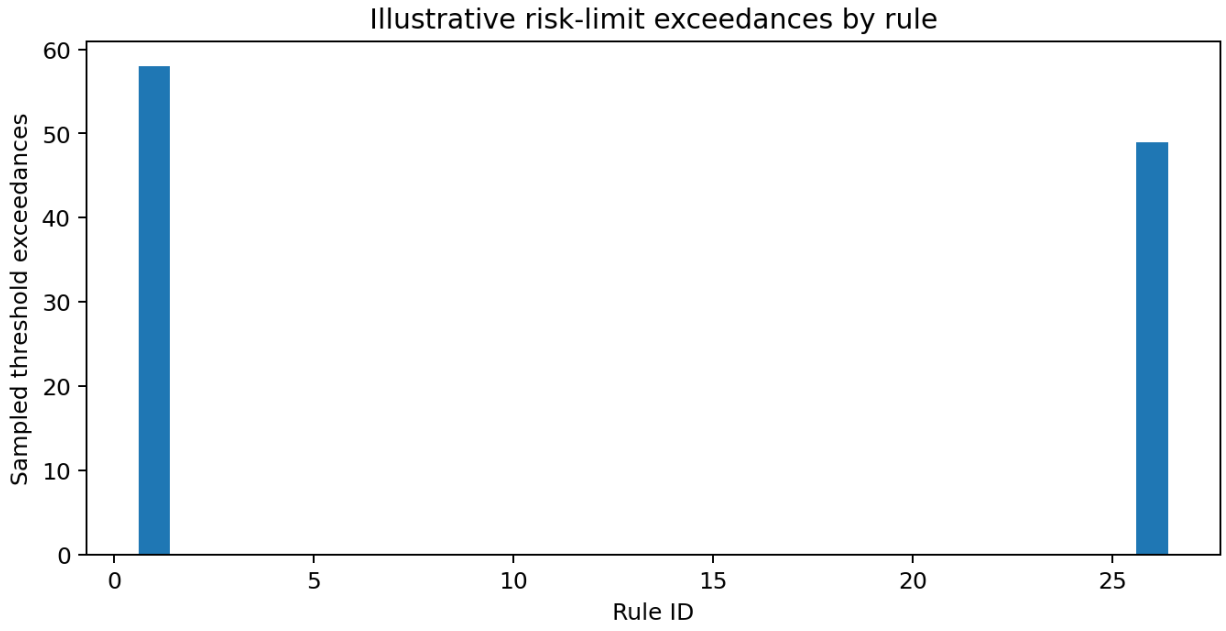


Figure. Host-calculated illustrative risk-limit events across formula variants.

9. Research interpretation and references

Demonstrated. Fifty variant mathematical expressions, controlled immutable changes, repeated evaluations on synthetic intraday exposures, scalar and batch invocation of the original paceval SDK, and a two-factor 3D analytical risk response are all executed and independently checked.

Not established. This is not a production market-risk monitoring platform or a backtested/approved VaR, expected shortfall, stressed ES, or Basel capital calculation. Covariance estimation, non-Gaussian tail losses, nonlinear derivative valuations, liquidity horizons and market-data integrity are not modelled. Results do not constitute investment advice.

Reproduction from the GitHub package

The C++ target builds using src/CMakeLists.txt with a compatible paceval Linux shared library supplied externally. Run intraday_risk with --updates 20000 and --out, then use src/verify_results.py, src/high_precision_reference.py, src/make_figures.py, src/make_whitepaper.py and src/check_readme_math.py. All released scientific graphs are regenerated from the supplied examples/ CSVs.

Selected research and industrial references

RiskMetrics, Technical Document, 4th edition (1996), www.msci.com/documents/10199/5915b101-4206-4ba0-aee2-3449d5c7

BIS, Basel Framework MAR30, Internal models approach: general provisions, updated 2024-07-05, www.bis.org/committees/bcbs/basel-framework/standard/mar/30/inforce/2023-01-01/published/2024-07-05

BIS, Basel Core Principles BCP40, published 2024-04-25, www.bis.org/committees/bcbs/basel-framework/standard/bcp/40/inforce/2024-04-25/published/2024-04-25

Reserve Bank of Australia, Measuring Traded Market Risk: VaR and Backtesting Techniques (1997), www.rba.gov.au/publications/rdp/1997/9708/value-at-risk.html

paceval C/C++ API, version 4.25 documentation, paceval.com/api/